CV-Lab5实验报告

杨子涵 2400012732

2025 年 12 月 5 日

1 函数实现思路

1.1 LinearClassifier类

一个输入为in_channels,输出为out_channels的线性层。

1.2 FCNN类

网络结构为线性层与ReLU交叉出现。

1.3 train函数

这里有个比较值得说的是数据预处理部分使用了以下代码：

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465),
                         (0.2023, 0.1994, 0.2010))
])
```

这段代码在创建数据集的时候进行归一化。PyTorch官网给的教程是直接用 `(0.5, 0.5, 0.5), (0.5, 0.5, 0.5)` 归一化，而另外很多地方是用的几个神秘数字 `(0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)`。查阅资料发现后者是CIFAR-10数据集中所有图片的RGB三个通道的均值和标准差。所以这样就是利用数据集本身的统计量让它里面数据的分布更加均匀（比如蓝色通道原来普遍偏暗，归一化之后模型就不会偏重某个通

道）。写一个代码测试下我们下载下来的CIFAR数据集的均值和方差是不是真的是这几个数（如图1所示），结果果然如此。

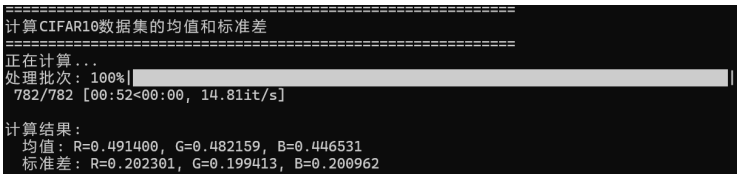


图 1: CIFAR-10数据集RGB通道的均值和标准差计算结果

2 开始训练我们的模型：Linear

首先我测试了用CPU训练和用GPU训练的速度差别。我电脑的CPU是13th Gen Intel® Core™ i9-13900HX，独显是NVIDIA GeForce RTX 4060 Laptop GPU。

用CPU和GPU运行一次对Linear分类器的训练，可视化结果如下图2所示。结果有点神奇，用GPU训练只比用CPU训练快了大约一倍。

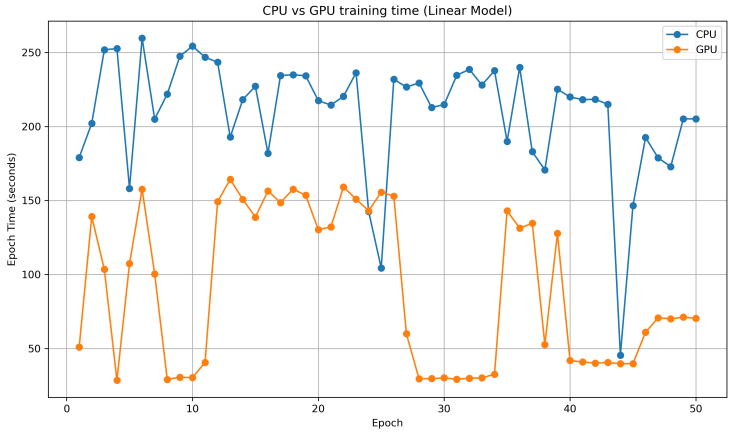


图 2: CPU与GPU训练速度对比

这可能是由于：

1. 我们训练的LinearClassifier就是一个in_channels到out_channels的全连接层，参数量只有$3072 \times 10 + 10$也就是三万多个，参数量较小。

2. CIFAR-10数据集分辨率为$32 \times 32 \times 3$，我设置的batch=128。所以单次用于计算的数据很少。

3. ToTensor, Normalize这些数据预处理全部由CPU主线程做，所以GPU会有相当的时间在等待数据。

运行nvidia-smi查看显卡占用情况，发现占用率不到20%，说明以上猜测是可能的。

其次，

```
trainloader = DataLoader(trainset, batch_size=128,
                         shuffle=True, num_workers=0)
testloader = DataLoader(testset, batch_size=100,
                        shuffle=False, num_workers=0)
```

这两行代码里的num_workers也是一个可以选择的参数，代表并行的进程数。按道理说，num_workers越大则训练速度越快，因为刚刚提到了GPU在训练的同时CPU在准备数据，那么使用多线程就会加快CPU这边处理的速度。实际测试得到的数据可视化结果如下图3所示，结果很神奇，num_workers越大则训练时间越长！

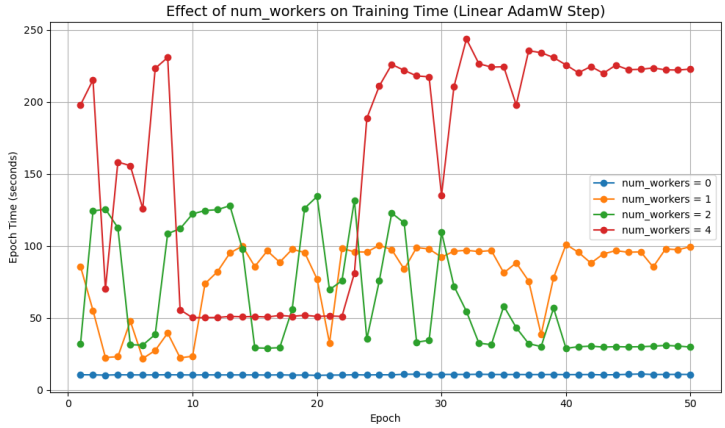


图 3: 不同num_workers设置下的训练时间对比

询问了一下AI，发现这是因为Windows不支持POSIX fork，而是靠spawn创建子进程，这需要从头启动新的python进程并且重新import所

有模块；而Linux的fork几乎是O(1)操作（写时复制COW），每个worker都共享父进程内存，启动非常快。并且我们的CIFAR-10数据集很小，这导致进程间通信的时间比加载数据的时间还长。而num_workers=0时只有一个主进程在工作，所以每个epoch的时间很稳定，这也可以从图上看出来，符合我们的预期。

以下是训练Linear时tensorboard上记录的曲线（如图4所示），optimizer为AdamW，scheduler为Step。最终在测试集上的准确率为39.77%。

a Learning Rate　b Test Accuracy　c Test Loss　d Train Accuracy　e Train Loss

图 4: Linear分类器训练过程中的TensorBoard曲线

之后的训练都使用GPU，num_workers都设置为0。

3　开始对FCNN的训练

我一开始选取的网络结构为：

```
self.network = nn.Sequential(
    nn.Linear(in_channels, hidden_channels),
    nn.ReLU(),
    nn.Linear(hidden_channels, hidden_channels // 2),
    nn.ReLU(),
    nn.Linear(hidden_channels // 2, out_channels)
)
```

tensorboard上记录的Learning rate如图5所示。

可以看到step的LR是在epoch为20,40的时候突变为原来的0.1，而cosine的LR是按余弦变化，这符合我们的预期。

在此网络结构，epoch=50的条件下，tensorboard上记录的 test accuracy、test loss、training accuracy、training loss 曲线如下所示(图6、7、8和9)。

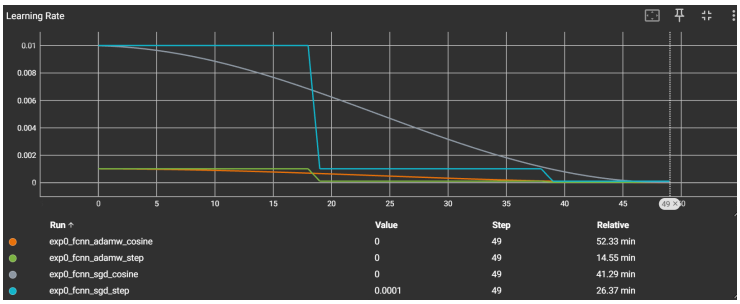


图 5: Learning Rate变化曲线

图 6: Test Accuracy曲线

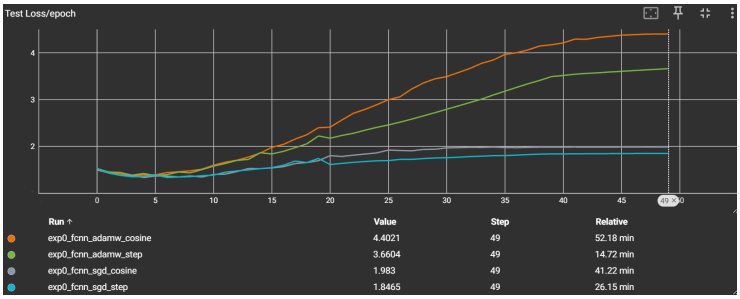


图 7: Test Loss曲线

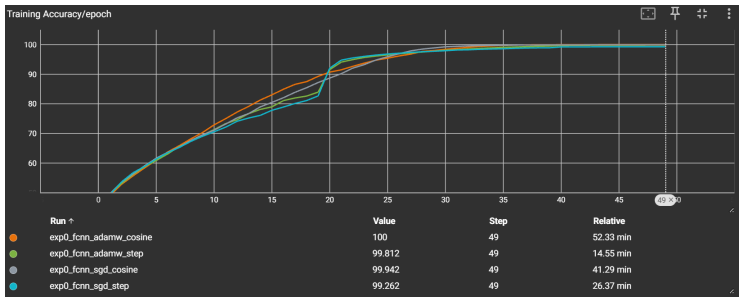


图 8: Training Accuracy曲线

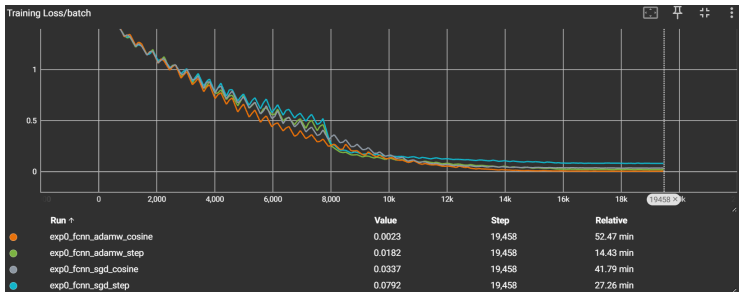


图 9: Training Loss曲线

3.1 对比adamw和sgd

可以看到，training accuracy曲线和training loss曲线中adamw和sgd的表现接近，说明在CIFAR-10这个任务中二者都能让模型贴合数据集。但是在test accuracy和test loss曲线中可以看出sgd的test accuracy明显大于adamw，test loss明显小于adamw。

3.2 对比step和cosine

可以看到，test accuracy曲线中，不论是用两种优化器中的哪一种，稳定之后（epoch=40）cosine训出来的模型都会比step略高一点点（不到0.5%）。但是在训练过程中step曲线的最高点并没有比cosine低，这表明step在训练到达一定轮次后受到过拟合的影响较大。

不过从最终准确率、收敛速度等指标来看，step和cosine的效果差距不大。

另外，从图中可以看出使用step作为scheduler时test accuracy、training accuracy在epoch=20时有突然提升， training loss在epoch=20的时候有突

然下降，而test loss下降不明显。下面是所有用step作为scheduler的曲线，可以看到，这确实是一个共通点。

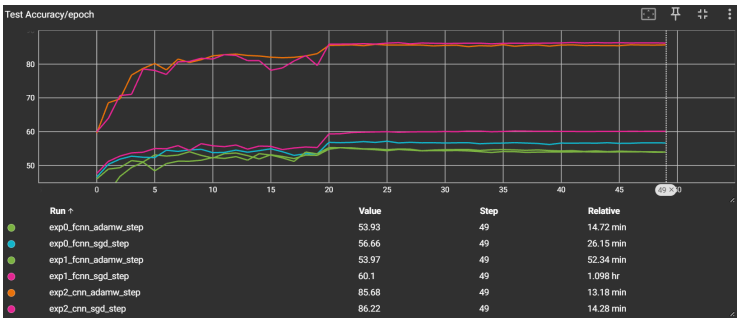


图 10: Test Accuracy曲线（step scheduler）

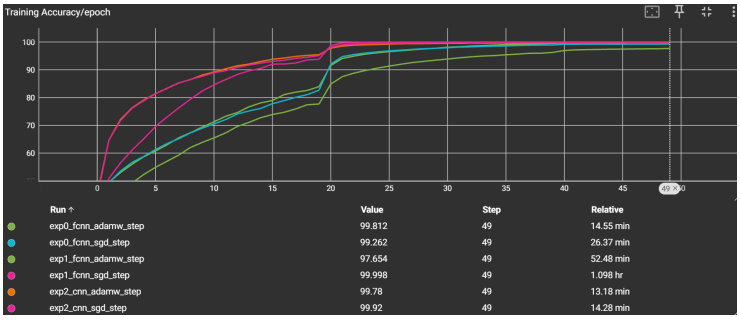


图 11: Training Accuracy曲线（step scheduler）

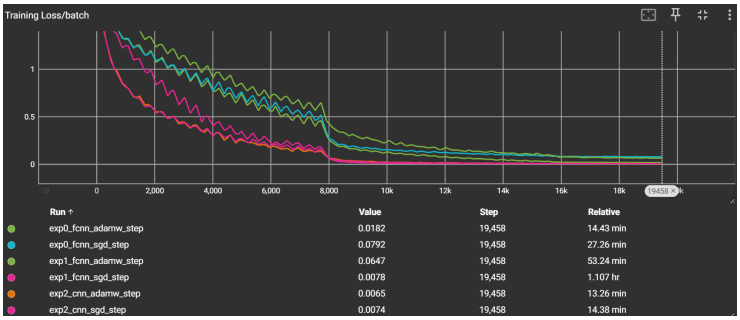


图 12: Training Loss曲线（step scheduler）

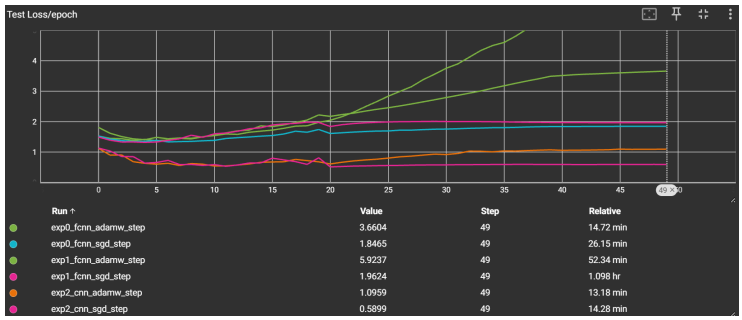


图 13: Test Loss曲线（step scheduler）

这可能是由于LR比较大的时候每一步太大了，所以就很难精准停在最低点附近，而lr变小之后的第一个时刻，相比于上一个时刻一定是能更精确地向着最优处走的，所以accuracy会突然提升。Loss是连续的指标，它不像accuracy那样是从0直接改变到1，所以变化幅度相对没有那么大。而Training loss下降幅度比Test loss大可能是因为test loss不是直接受梯度控制，只是受模型“变强”影响，所以不会突然下降。

为了验证如上分析的正确性，我更改了网络结构后又做了一组实验。其中所使用的网络结构如下所示：

```
self.network = nn.Sequential(
    nn.Linear(in_channels, 64 * hidden_channels),
    nn.ReLU(),
    nn.Linear(64 * hidden_channels, 4 * hidden_channels),
    nn.ReLU(),
    nn.Linear(4 * hidden_channels, hidden_channels // 4),
    nn.ReLU(),
    nn.Linear(hidden_channels // 4, out_channels)
)
```

在该网络结构、epoch=50 的条件下，tensorboard 上记录的 test accuracy、test loss、training accuracy、training loss 曲线如下所示。（图14、15、 16和17）

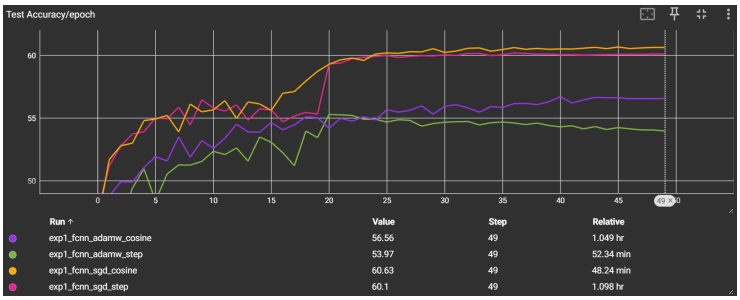


图 14: Test Accuracy曲线

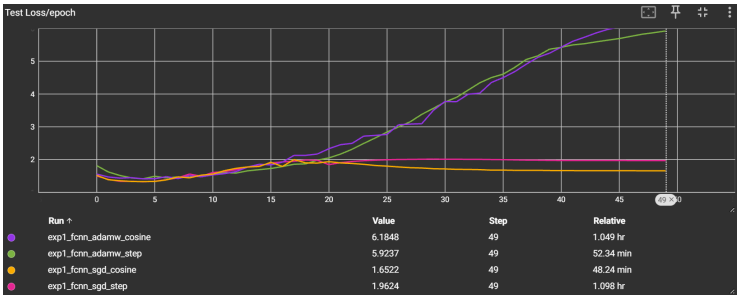


图 15: Test Loss曲线

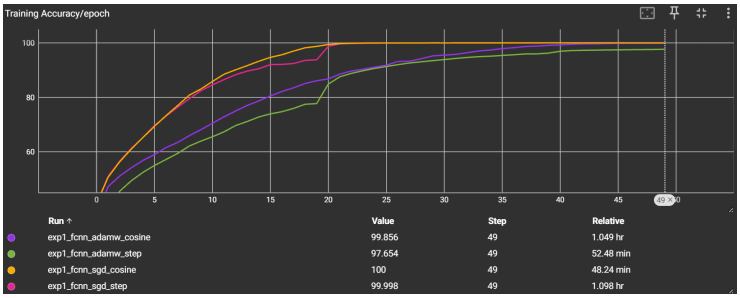


图 16: Training Accuracy曲线

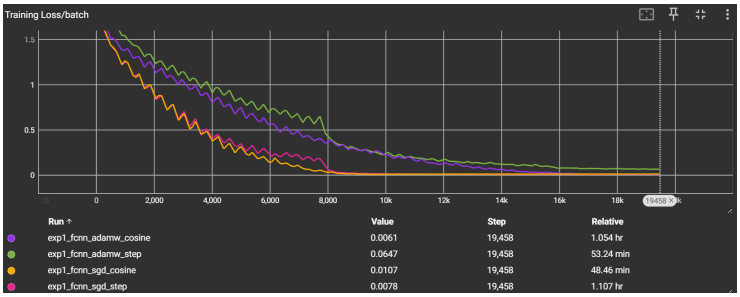
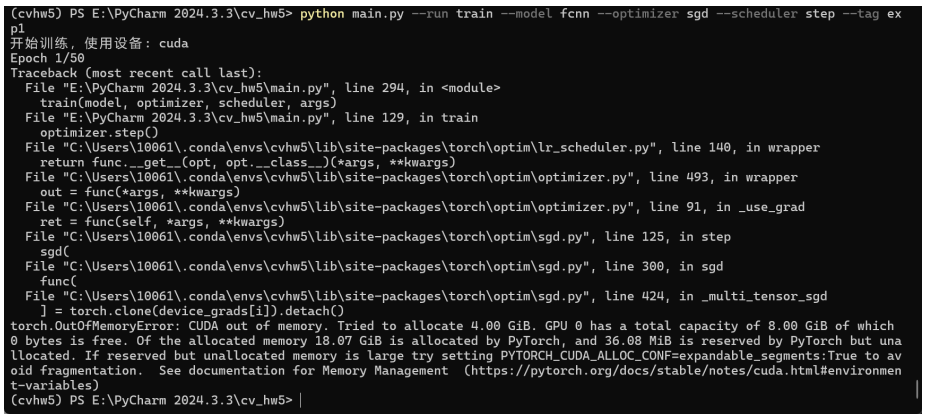


图 17: Training Loss曲线

可以看到，这一组实验的结果依旧符合我们刚刚的分析。

4 选取最优的超参数

一般来说，提高模型正确率最简单粗暴的方法就是增大网络规模。然而，当我尝试加入一个 (in_channels, 1024*hidden_channels) 的线性层时，程序直接出现经典的 CUDA out of memory（悲），如图 18 所示。



```
(cvhw5) PS E:\PyCharm 2024.3.3\cv_hw5> python main.py --run train --model fcnn --optimizer sgd --scheduler step --tag ex
p1
开始训练，使用设备：cuda
Epoch 1/50
Traceback (most recent call last):
  File "E:\PyCharm 2024.3.3\cv_hw5\main.py", line 294, in <module>
    train(model, optimizer, scheduler, args)
  File "E:\PyCharm 2024.3.3\cv_hw5\main.py", line 129, in train
    optimizer.step()
  File "C:\Users\10061\.conda\envs\cvhw5\lib\site-packages\torch\optim\lr_scheduler.py", line 140, in wrapper
    return func.__get__(opt, opt.__class__)(*args, **kwargs)
  File "C:\Users\10061\.conda\envs\cvhw5\lib\site-packages\torch\optim\optimizer.py", line 493, in wrapper
    out = func(*args, **kwargs)
  File "C:\Users\10061\.conda\envs\cvhw5\lib\site-packages\torch\optim\optimizer.py", line 91, in _use_grad
    ret = func(self, *args, **kwargs)
  File "C:\Users\10061\.conda\envs\cvhw5\lib\site-packages\torch\optim\sgd.py", line 125, in step
    sgd(
  File "C:\Users\10061\.conda\envs\cvhw5\lib\site-packages\torch\optim\sgd.py", line 300, in sgd
    func(
  File "C:\Users\10061\.conda\envs\cvhw5\lib\site-packages\torch\optim\sgd.py", line 424, in _multi_tensor_sgd
    ] = torch.clone(device_grads[i]).detach()
torch.OutOfMemoryError: CUDA out of memory. Tried to allocate 4.00 GiB. GPU 0 has a total capacity of 8.00 GiB of which
0 bytes is free. Of the allocated memory 18.07 GiB is allocated by PyTorch, and 36.08 MiB is reserved by PyTorch but una
llocated. If reserved but unallocated memory is large try setting PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True to av
oid fragmentation.  See documentation for Memory Management  (https://pytorch.org/docs/stable/notes/cuda.html#environmen
t-variables)
(cvhw5) PS E:\PyCharm 2024.3.3\cv_hw5>
```

图 18: CUDA out of memory!

最终，我选择的网络结构如下所示：

```
self.network = nn.Sequential(
    nn.Linear(in_channels, 64 * hidden_channels),
    nn.ReLU(),
    nn.Linear(64 * hidden_channels, 4 * hidden_channels),
    nn.ReLU(),
    nn.Linear(4 * hidden_channels, hidden_channels // 4),
    nn.ReLU(),
    nn.Linear(hidden_channels // 4, out_channels)
)
```

训练时使用的超参数设置如下：

- optimizer: SGD
- scheduler: cosine

- learning rate: 0.01
- momentum: 0.9
- weight decay: 0.001
- epoch: 50
- batch size: 128

在上述设置下训练得到的模型在测试集上的准确率为 **60.63%**（如图14所示），满足作业要求。

5 尝试训练一个CNN

我还尝试了训练一个基于 CNN 的分类模型。其网络结构如下：

```
self.conv_layers = nn.Sequential(
    # input:[batch_size, 3, 32, 32]
    nn.Conv2d(3, 32, kernel_size=3, padding=1),
    nn.BatchNorm2d(32),
    nn.ReLU(),
    nn.Conv2d(32, 32, kernel_size=3, padding=1),
    nn.BatchNorm2d(32),
    nn.ReLU(),
    nn.MaxPool2d(2),

    nn.Conv2d(32, 64, kernel_size=3, padding=1),
    nn.BatchNorm2d(64),
    nn.ReLU(),
    nn.Conv2d(64, 64, kernel_size=3, padding=1),
    nn.BatchNorm2d(64),
    nn.ReLU(),
    nn.MaxPool2d(2),

    nn.Conv2d(64, 128, kernel_size=3, padding=1),
    nn.BatchNorm2d(128),
```

```
        nn.ReLU(),
        nn.Conv2d(128, 128, kernel_size=3, padding=1),
        nn.BatchNorm2d(128),
        nn.ReLU(),
        nn.MaxPool2d(2),
    )

    self.fc_layers = nn.Sequential(
        nn.Flatten(),
        nn.Linear(128 * 4 * 4, 512),
        nn.ReLU(),
        nn.Dropout(0.5),
        nn.Linear(512, 256),
        nn.ReLU(),
        nn.Dropout(0.5),
        nn.Linear(256, out_channels)
    )
```

如图 19–22 所示，训练结果如下：

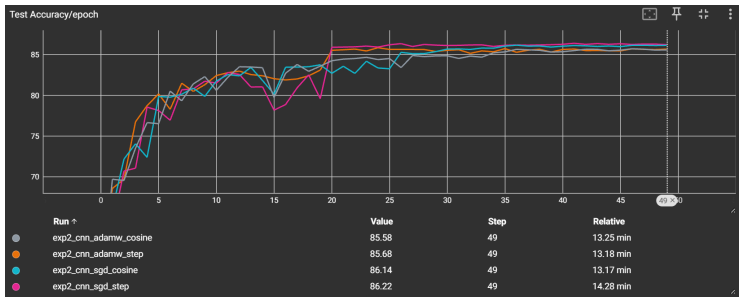


图 19: Test Accuracy曲线

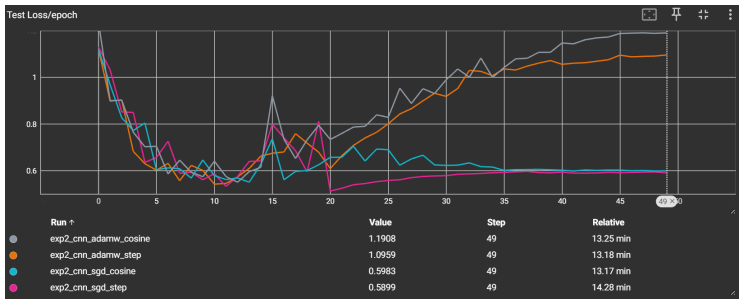


图 20: Test Loss曲线

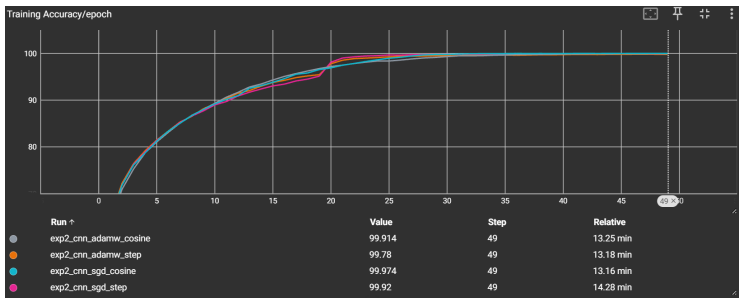


图 21: Training Accuracy曲线

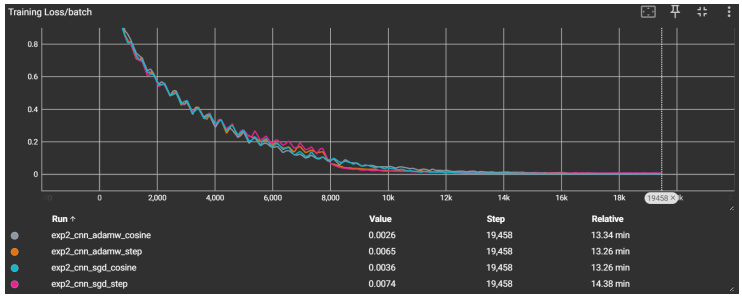


图 22: Training Loss曲线

可以看到，在第二个 epoch 时模型的准确率就超过了 60%，并且在训练 50 个 epoch 后准确率达到了 85% 以上。这说明在处理图像任务时，CNN 模型确实具有非常强的表现力。

另外，这一组结果也与我们在对比 adamw/sgd、step/cosine 那一小节的分析相符合，说明我们的分析具有一定普适性。